PipedPeer: Running Unmodified Python Across a Small Group's Machines Without a Configured Cluster

Yateen Vaviya[a], Vinayak Yadav[a], Sahil Gupta[a], Rupali Sawant[a], Dayanand Ambawade[a]

[a]*Department of Computer Engineering, Sardar Patel Institute of Technology, Mumbai, Maharashtra, India*

Abstract

Cluster frameworks such as Spark, Ray and Dask assume a configured cluster, a program written against their interface, and machines that can reach one another. These assumptions put the idle laptops, desktops and phones of a small group out of reach, particularly when the machines sit behind different home or campus routers. PipedPeer runs an unmodified Python script on other machines. Every machine runs the same daemon and there is no head node. The script's environment is built as a Nix closure pinned to one package-set revision, and a peer receives only the store paths it lacks. Parallel maps, joblib, pandas grouping and joins, NumPy products and PyTorch data-parallel training are substituted at import time. A map is divided by a makespan model fitted to measured peer rates, and numerical work moves only when a cost model predicts a gain. Machines on different networks meet through an introducer that every daemon serves and then connect over direct, authenticated QUIC links; no job data is relayed. We evaluate PipedPeer in container labs, on two laptops in different homes with a phone, and on a second pair of machines. Interception raises the startup of a trivial job by 1.24×, and a repeated submission whose closure the worker holds takes 0.83 s against 23.4 s for the first. A mixed CPU script took 13.3–14.6 s on two laptops against 17.4–18.1 s on one. Periodic weight averaging every 32 steps kept validation loss within 1% of one device on three test models. Small

Email addresses: yateen.vaviya23@spit.ac.in (Yateen Vaviya), vinayak.yadav23@spit.ac.in (Vinayak Yadav), sahil.gupta23@spit.ac.in (Sahil Gupta), rupali_sawant@spit.ac.in (Rupali Sawant), dd_ambawade@spit.ac.in (Dayanand Ambawade)

training jobs were not faster on two machines than on one GPU. The evaluation covers two or three physical machines and is not a controlled scaling study.

Keywords: distributed computing, peer-to-peer systems, NAT traversal, transparent interception, reproducible environments, heterogeneous scheduling, data-parallel training

---

1. Introduction

The gap between running `python script.py` on one machine and running the same work on several is large in operational effort and often small in the computation involved. Apache Spark [1], Ray [2] and Dask [3] presuppose a configured cluster with a head node, matching software on every worker, a scheduler, and a network on which every machine reaches every other. Remote execution over `ssh` avoids the setup but offers no environment management, placement, isolation or fault tolerance.

For a small group — a student team, a laboratory, a household — three obstacles stand between a script and the machines around it, and they are usually treated separately:

1. *Environment*: a remote machine must be made to hold the same interpreter and libraries as the local one.
2. *Code*: using more than one machine normally means rewriting the program against a framework's API.
3. *Reachability*: machines behind different home or campus routers accept no inbound connections, and the usual remedies — a VPN, port forwarding or a relay — are either configuration or a bottleneck.

PipedPeer addresses all three for Python on Linux. A user runs `pipedpeer run script.py`; the client builds the script's environment as a Nix closure [12], picks a machine, ships the missing store paths and the workspace, and runs the job in a sandbox. A Python package imported before the script's first line substitutes the parallel primitives the script already uses, and spreads their work across machines when a model of the cluster predicts that doing so is faster. Machines on different networks join with one invite code and connect directly.

Contributions.

1. **Peer-to-peer closure distribution** (Section 4.2): signed closures pushed between peers, diffed by store path, with distributed work offered only to peers that can run the closure, and environments of a foreign architecture built by the receiver.
2. **Import-time interception with measured work sharing** (Sections 4.4–4.6): parallel maps divided by a makespan model over measured rates, with speculative copies of late parts, and numerical work gated by a cost inequality.
3. **Joining across networks without a relay** (Section 4.3): an introducer embedded in every daemon that only exchanges addresses, router port mapping, a candidate race on one shared socket, and a birthday-collision fallback.
4. **Placement without a scheduler process** (Section 4.1): filters and a bounded score computed in the submitting client, with a three-phase lease.
5. **Speed-weighted data-parallel training through the daemons** (Section 4.7), with exact or periodic-averaging synchronisation and an interval chosen per ring size from accuracy measurements.
6. **An evaluation that keeps emulated, laboratory and physical-machine results apart** (Section 6), including the cases in which distribution was slower than one machine.

Scope. PipedPeer targets two to a handful of machines owned by one group that trusts itself. It is not a replacement for the frameworks above at scale, and the evaluation does not establish speedup beyond three machines (Section 8).

2. Background and related work

Cluster frameworks. Spark [1], Ray [2] and Dask [3] are mature and far more capable at scale. Ray ships a drop-in `multiprocessing.Pool` and a `joblib` backend, and Modin [4] provides distributed pandas behind the pandas API; each needs a changed import and a running cluster. Kubernetes [5] and HTCondor [6] schedule whole jobs through a central manager and explicit job descriptions.

Drop-in parallelism for Python. Several systems keep the `multiprocessing` interface and change only where the work runs. Fiber [7] replaces `multiprocessing` with processes on a cluster manager; PyWren [8] and its successor Lithops [9] run maps on serverless functions, and Lithops exposes a `multiprocessing`-compatible API. Parsl [10] and funcX [11] turn decorated Python functions into tasks on remote resources. All of these require an import or decorator change and a provisioned back end (a cluster manager or a cloud account). PipedPeer's substitution happens before the script's first statement, so no line changes, and it decides per call whether to distribute at all. The closest systems in intent are therefore Fiber and Lithops; the differences are that PipedPeer needs no back end beyond the peers themselves, ships the environment, and may decline to distribute.

Reproducible environments. Nix [12, 13] and Guix [14] name each build by a hash of its inputs, which makes a store path a stable name for an environment; Spack [15] targets HPC software stacks; container images [16, 17] ship a whole filesystem. PipedPeer uses Nix store paths and Nix's own binary cache format, and adds peer-to-peer pushing, path-level diffs between peers, and a rule that only peers able to run the closure receive distributed work.

Harvesting idle machines. Condor [6] scavenged idle workstations; SETI@home [18] and BOINC [19] harvest volunteers' machines for specific projects. Smartphones have been studied both as offload clients — MAUI [20] and CloneCloud [21] move work *off* a phone — and as compute donors: CWC [22] schedules jobs on phones while they charge, and Honeybee [23] distributes work among nearby phones. PipedPeer's phone support is narrower than either line: an Android phone under Termux takes parallel-map items (Section 4.9).

NAT traversal. Reflexive addresses are discovered as in STUN [24]; port mappings are requested with PCP [25], NAT-PMP [26] and UPnP-IGD [27]; ICE [28] standardises candidate gathering and pairing; simultaneous UDP probing from the socket that will carry the connection is the hole punching of Ford et al. [29]; and the birthday-collision technique for a port-randomising NAT follows Anderson [30]. Links are QUIC [31]. Overlay networks such as Tailscale, which runs over WireGuard [32], solve the same problem with a coordination service and relays; PipedPeer embeds the coordination in its own daemons and never relays job data.

Scheduling on heterogeneous machines. Dividing divisible work among machines of different speeds to minimise makespan is classical scheduling on uniform machines [33, 34]. MapReduce's backup tasks [35] and LATE [36] launch speculative copies of stragglers; PipedPeer applies the same idea to the parts of one parallel map. Edge-computing work studies the offloading decision our cost model makes: weighing transmission against local computation [37], partial over all-or-nothing offloading [38], learned policies [39], and topology-aware placement [40], which outperforms the scalar scoring PipedPeer uses.

Data-parallel training over weak links. PyTorch DDP [41] and Horovod [42] use ring all-reduce among ranks that reach one another directly. Communication is reduced by 8-bit [43] and 1-bit [44] gradient compression with error feedback, by low-rank compression [45], and by synchronising less often: local SGD [46, 47], federated averaging [48] and DiLoCo [49]. Collaborative training over the internet has been demonstrated by Learning@home [50], DeDLOC [51] and Petals [52]. PipedPeer's training path is smaller in ambition than these: two to four ranks of an unmodified script, synchronised through the daemons, with the synchronisation interval chosen from accuracy measurements (Section 6.5).

Failure transparency. Spenger [53] defines failure transparency, under which application code never observes a failure. Scripts run by PipedPeer contain no failure handling, so every distributed path must fall back to a local one (Section 4.8).

Table 1 summarises the comparison. Its entries come from published designs, not from benchmarks.

3. Design overview

Goals. **(G1)** The script runs unchanged. **(G2)** Joining needs one setup command and, across networks, one invite code. **(G3)** No machine is special: there is no head node, scheduler or registry, and every machine both submits and works. **(G4)** Distribution never changes the result and is declined when the model predicts it would be slower. **(G5)** No job data passes through a third machine.

Table 1: PipedPeer against existing systems (qualitative). The lower block is where PipedPeer is partial or the weaker system.

Property	**PipedPeer**	**Spark**	**Ray**	**Dask**	**Lithops**	**ssh**
Runs unmodified Python	yes	no	partial	partial	partial	yes
No cluster or back end to configure	yes	no	no	no	no	partial
No head or master node	yes	no	no	no	n/a	yes
Reproduces the environment	yes	partial	partial	partial	partial	no
Joins NATed machines, no relay†	yes	no	no	no	n/a	no
Survives losing a node*	partial	yes	yes	yes	yes	no
Authenticated, encrypted	partial	yes	partial	partial	yes	yes
Sandboxes each job	partial	no	no	no	yes	no
Runtimes beyond Python	no	yes	partial	no	no	yes
Proven in production	no	yes	yes	yes	yes	yes

†When one introducer is reachable; otherwise a public address or an overlay network is needed. *Any worker; not the submitter of a parallel script nor the machine of a training ring's rank 0.

Architecture. Every machine runs a single daemon; the command-line client is short-lived (Fig. 1). The coordinator is a library linked into the client, so placement is decided where the job is submitted. Every daemon also serves the introducer for its own cluster, and a training ring's lead is whichever node runs rank 0. On one network participants find each other by broadcast; across networks a participant runs `pipedpeer connect` with a one-line code printed by another member's `pipedpeer invite`, which carries the cluster secret and up to six introducer addresses with a checksum.

Life of a job. (1) The client scans the script's imports and builds or reuses its closure. (2) It places the job and leases a slot on the chosen worker. (3) It sends only the store paths the worker lacks, then the workspace. (4)

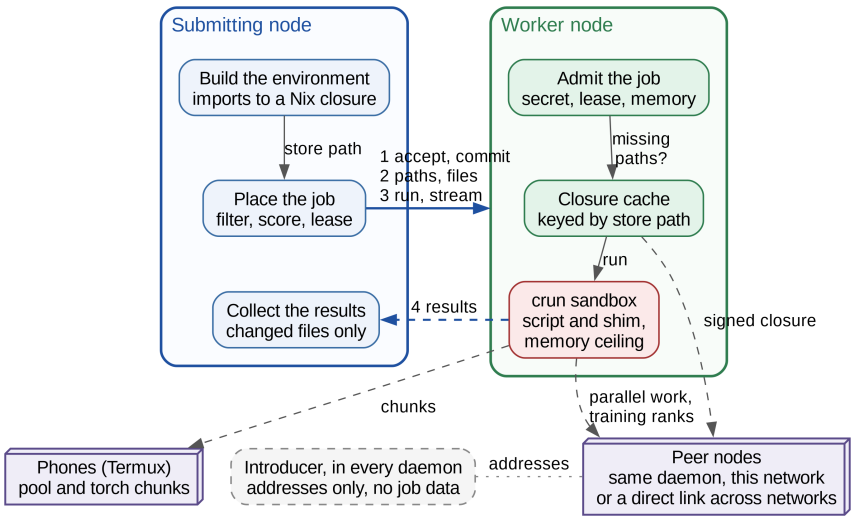


Figure 1: System design. Submitter and worker are the same binary. Peers on other networks are reached over direct links that the introducer helps set up but never carries. Phones take parallel-map and torch chunks only.

The worker runs the script in a sandbox with the interception package on the interpreter's path. (5) Parallel calls inside the script may be divided among peers that can run the same closure. (6) Output is streamed back and changed files are synchronised to the submitter.

4. Mechanisms

4.1. Placement

Candidates come from known peers, LAN discovery and internet-linked peers. Hard filters run in order — architecture, GPU if required, then free memory, cores and VRAM — and the remaining candidates are scored:

$$S = \Big(1 - \frac{u_{\text{cpu}} + u_{\text{mem}}}{200} - 0.05\, n_{\text{jobs}} + 0.10(\hat{c} - 0.5) + 0.06(\hat{f} - 0.5) + 0.10(\hat{m} - 0.5) + 0.10(\hat{r} - 0.5)\Big) \cdot h, \tag{1}$$

with utilisations u in percent, n_{jobs} running jobs, capacity terms $\hat{c} = \min(\text{cores}/32, 1)$ and $\hat{f} = \min(\text{MHz}/4000, 1)$, headroom terms

$\hat{m} = \min(\text{free GiB}/32, 1)$ and $\hat{r}$ (free fraction of cores), 0.5 for a missing metric, and h the stored health score (1.0 for a node that answers). S is clamped at zero; a GPU request adds up to 0.8 from VRAM, device count and compute capability. The weights are hand-set: the utilisation terms can subtract up to 1.0 and the four normalised terms move S by at most 0.18 either way, so current load dominates hardware. We did not run a sensitivity analysis of these weights (Section 8).

Free memory is what a node can *grant*: where a user `systemd` is available, each daemon runs under a cgroup ceiling of half the machine, and daemons co-located on one machine subtract one another's reservations. Work is reserved by a three-phase lease (accept, commit, complete); the concurrency and memory checks and the insertion happen under one lock. The submitter renews every 30 s, also while the environment builds and uploads, against a 5-minute expiry. A submitter that dies stops renewing and its slot is reclaimed; a worker that dies has its job rescheduled with back-off.

4.2. Environment reproduction

The client resolves the script's imports to packages and builds a Nix closure from a flake pinned to an *exact* package-set revision, so the store path is a stable name for an environment across machines and time. Imports absent from the mapping table are looked up by name in the pinned set, the scan follows the script's own modules, a `requirements.txt` is honoured, and an import that is still unresolved is reported before the run.

Before uploading, the client asks whether the target can already run the closure and otherwise sends only the store paths it lacks. The worker then pushes the closure to peers that cannot yet run it, since only peers that can are offered distributed work. Between daemons the push is a binary cache produced and imported by Nix itself and signed with the sending node's key; the receiver checks that key against the signer's published identity before trusting it for that import. The upload returns once the target holds the closure; pushes to slower peers continue in the background and do not hold up the run.[1] Closures go only to peers of the same architecture; a peer

[1] An earlier build made the upload wait on the synchronous push to every peer. In an emulated three-node lab this blocked a cold run for about 600 s and, because a peer still importing was asked again, a warm run too (579 s). The behaviour described here replaced it (Section 6.3).

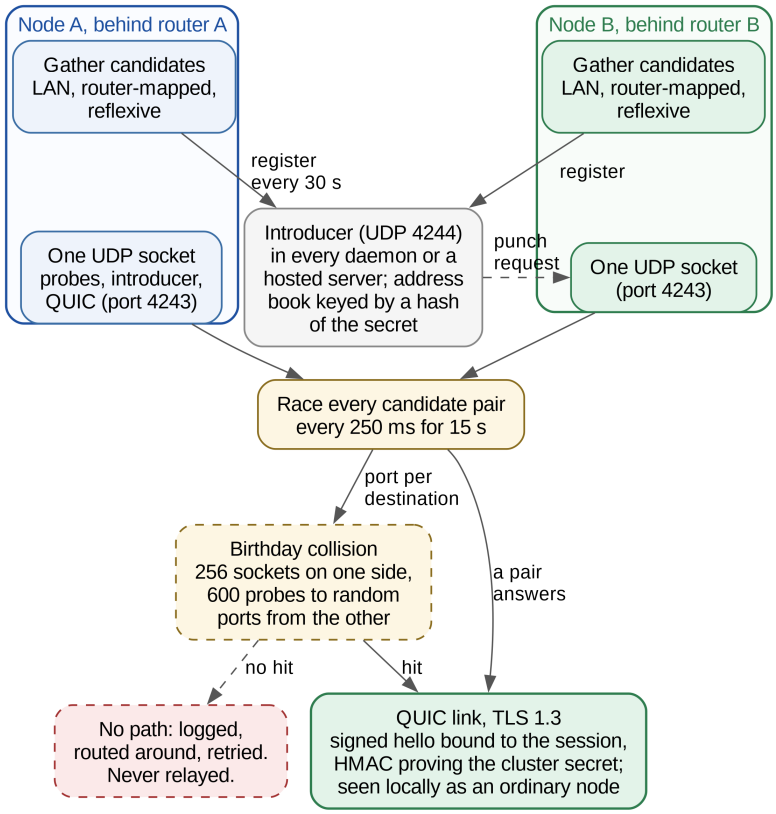


Figure 2: Joining across networks. The introducer exchanges candidate addresses and forwards one punch request; the two nodes then race their candidate pairs over one UDP socket.

of another architecture receives the job's package list and builds its own environment once, caching it by flake hash.

4.3. Joining across networks

An introducer keeps an address book keyed by a hash of the cluster secret and forwards punch requests between two nodes; it carries no job data. Every daemon serves one for its own cluster and registers with every introducer it knows, so no single machine's address is a point of failure. Each node offers its LAN addresses, a port mapped on its router (published only when the router's external address matches the one the introducer observed, which excludes a router behind a second NAT) and its reflexive address (Fig. 2).

Probes, introducer messages and QUIC share one UDP socket, because a router's mapping belongs to the socket that opened it; both sides probe every candidate pair every 250 ms for 15 s rather than trying candidates in turn.

When one side's NAT allocates a fresh external port per destination, there is no address to publish. That side opens 256 sockets towards the peer while the other sends 600 probes to random ports among the 64,511 above 1024, and the socket that is hit is kept [30]. If the NAT draws ports uniformly, a round succeeds with probability

$$P = 1 - \left(1 - \frac{256}{64511}\right)^{600} \approx 0.91. \tag{2}$$

This is a computed figure under that assumption; we did not measure a per-round success rate. A container lab with two cone and two port-randomising translators exercises the fallback (cone–randomising pairs are expected to link through it, and a pair of two randomising NATs to be reported unreachable), and the four defects it exposed — among them a socket found by a collision that stopped listening once adopted — are fixed.

The winning path carries QUIC with a hello signed by each node's Ed25519 [55] key and checked against the node it meant to reach. The hello signs the session's exported keying material [56] and proves the cluster secret by an HMAC [57] over it, so it cannot be replayed on another session. The link appears locally as a loopback forwarder recorded as one path to that node, so placement, transfer and gradient exchange run over it unchanged. A node is identified by its key's fingerprint, never by its address: the node store keeps every path to a node and switches only to a path that answered and proved the key, and QUIC connections migrate when an address changes. A peer with no path is reported with its reason and routed around, never relayed. The daemon API requires the cluster secret on every route and serves pinned TLS [58] and HTTP on one port.

4.4. Transparent interception

A Python package embedded in the binary is placed on the interpreter's path so that it is imported before the user's first line, and libraries are patched on their first import rather than at startup (Fig. 3). Substituted primitives are `multiprocessing` and `concurrent.futures` maps, `joblib`, pandas grouping, joining and chunked reading, NumPy matrix products and

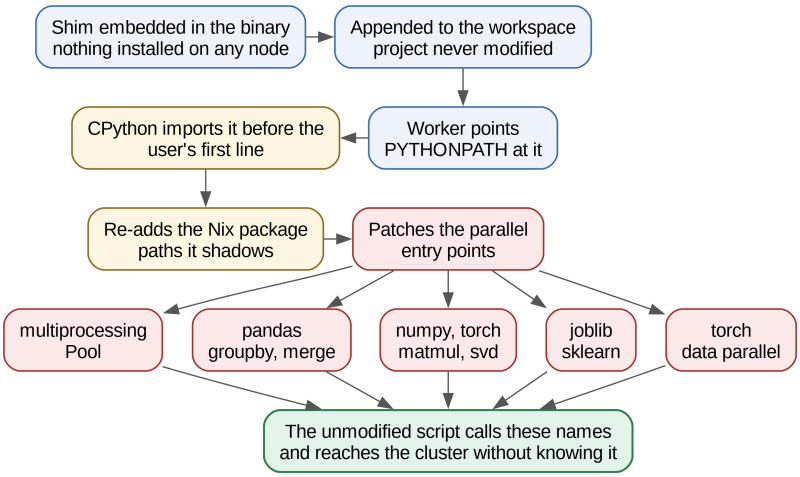


Figure 3: Interception. The package is placed on the interpreter's path so that it is imported before the script; libraries are patched on first import.

decompositions, and PyTorch data-parallel training. Every substituted call falls back to the original on any exception.

A parallel-map kernel travels as *source* when it is a module-level function, including one defined in `__main__`, or *by value* through cloudpickle [59] when it is a lambda or closure; a kernel that cannot be serialised runs on local threads. pandas grouping and joining hash rows by key, so each key is complete on one node and any aggregation is exact without a combiner; the row order of a join is not preserved.

4.5. Dividing a parallel map

The node that holds a map splits it once among its peers, and each peer runs its part on its own cores; a part is never split across machines again. Each device i (a node's CPU pool or one GPU) is modelled by a setup time o_i and an effective rate

$$r_i = \left(\frac{1}{\lambda_i} + \frac{\beta}{B_i} \right)^{-1}, \tag{3}$$

where λ_i is its measured throughput in items per second, β the bytes per item and B_i the measured bandwidth to it ($r_i = \lambda_i$ for the local device). A share of w_i items finishes at $o_i + w_i / r_i$. For W items, the makespan T is the

Algorithm 1 Dividing W items among devices

$D \leftarrow$ devices with a measured rate and enough memory for the working set
repeat
    $T \leftarrow$ root of $\sum_{i \in D} \max(0, T - o_i)\, r_i = W$
    $X \leftarrow \{\, i \in D : o_i \geq T \,\}$; $D \leftarrow D \setminus X$
until $X = \emptyset$
$w_i \leftarrow \lfloor (T - o_i)\, r_i \rceil$ for $i \in D$; return shares below a minimum to the others

root of

$$\sum_i \max(0,\, T - o_i)\, r_i = W, \tag{4}$$

which is piecewise linear and increasing in T and is solved by walking its breakpoints. A device with $o_i \geq T$ cannot start before the others finish and is dropped; dropping it raises T, so the rule is applied until the set settles (Algorithm 1). Rates and setup times are fitted from the parts each device has returned. Because the local device is always a candidate, the *predicted* makespan is never above the local one; the measured time can be (Section 6.4).

During execution a part that runs past its prediction counts as at least that long, and a part that is waiting or overdue is copied to a free device when the expected saving exceeds the copy's own setup (or 0.2 s); the first copy to finish is kept, in the manner of backup tasks [35, 36].

4.6. Cost model for numerical work

NumPy and pandas operations are gated. With ϕ operations and ρ link crossings per byte, bandwidth B, local and per-node remote throughput R and R', and K nodes holding the environment (the submitter included), divisible work moves only when its time per byte is lower on the cluster:

$$\frac{\phi}{R} \;>\; \frac{\rho}{B} \;+\; 1.3\,\frac{\phi}{R'K}. \tag{5}$$

Payload size cancels, so size enters as separate rules: at least two nodes and 32 MiB, and pandas work below 8 FLOP/B (grouping $\phi = 4$, joining $\phi = 2$, $R = 1$ GFLOP/s) stays local up to 512 MiB. A matrix product ($\phi = \max(8, n/8)$, $\rho = 5$, $R = 200$ GFLOP/s) that does not fit in memory moves

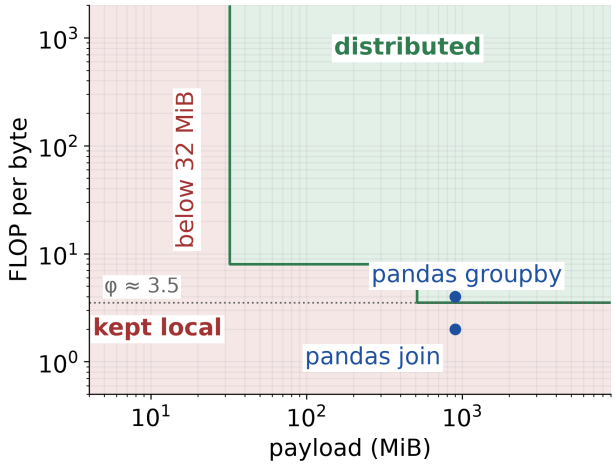


Figure 4: Decision regions of Eq. (5) for pandas work ($\rho = 1$, $R = R' = 1\,\mathrm{GFLOP/s}$) with three nodes on a 500 MB/s link. Above 512 MiB, work above about 3.5 FLOP/B is distributed: a 900 MiB grouping moves, a join of the same size stays local.

regardless, or raises an error if it fits nowhere. One GPU peer sets $R' = 6R$ for all K nodes. The factor 1.3, the GPU factor and the throughput constants are coarse, hand-set values that we did not calibrate across hardware. The test is first evaluated at 10 Gbit/s, and bandwidth is measured only if offloading could then win (Fig. 4). Indivisible work (singular value decomposition) moves when its transfer costs under half the local time. On the machines tested, local BLAS outruns a CPU cluster's transport, so dense products stay local.

4.7. Data-parallel training

Ranks post gradients to daemons rather than to one another (Fig. 5), which avoids the rank-to-rank reachability that ring all-reduce assumes. Each rank posts to the lead node's daemon; in exact mode on three or more directly reachable machines, it posts one shard to each rank's daemon. Every rank receives one sample-weighted average $\sum_i n_i g_i / \sum_i n_i$, sent as 8-bit values with per-block scales and error feedback [43, 44]. The global batch is partitioned across ranks in proportion to their measured speed, preserving the effective learning rate.

Unless the user fixed the ring size with `--ddp` N, after a warm-up the lead compares the split's step time with the fastest rank alone and keeps the ring only if it is faster. Admission takes the k fastest candidates maximising $k\,r_k$,

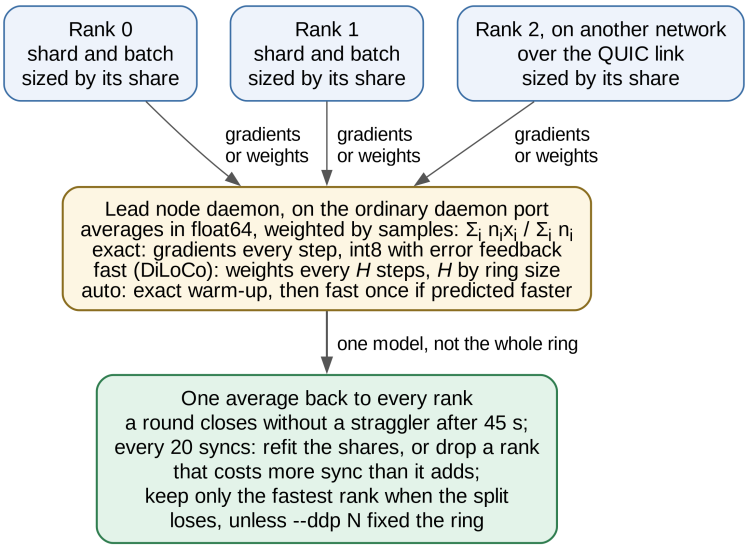


Figure 5: Training through the daemons. Ranks post gradients (or weights) to daemons rather than to one another and receive one sample-weighted average.

the equal-batch throughput; this is conservative, since placement cannot price synchronisation. Samples go to ranks by a weighted rendezvous hash [54]: when a rank dies the others take its share at the next epoch boundary with over 95% of their samples in place, and a returning rank joins from the lead's snapshot, taking about $1/n$ of them.

A fast mode synchronises every H steps: an exact warm-up (a tenth of the run, 10–500 steps), then local steps whose weights are averaged by samples, with optimiser moments averaged every tenth round; H shrinks to 1 over the run's last tenth. Fast mode never changes epochs, batch or learning rate, and must keep validation loss at most 1% above one device's (mean over seeds, on every test model). DiLoCo's outer Nesterov step (learning rate 0.7, momentum 0.9) multiplies each round's averaged change by $0.7/(1-0.9) = 7$ in the steady state; DiLoCo synchronises every few hundred steps, and at our intervals of 2–64 steps the amplified update failed the 1% bar, so PipedPeer uses plain averaging (outer learning rate 1, no momentum) and chooses H by ring size (Section 6.5). Automatic mode never picks fast mode beyond four ranks; an explicit fast ring above four runs at $H = 4$ with a warning. Under automatic synchronisation the lead switches a ring to fast mode once, when the predicted fast step with a 10% margin is shorter than the exact split's

and, for a ring not fixed by `--ddp` N, also shorter than the fastest rank's alone.

4.8. Degradation and operation

Using the cluster must not change the result. Parallel-map parts fill a result table in which the first result for each part is kept, idle local cores re-run any part not yet returned, and the call returns once every item is covered, so an unresponsive cluster degrades to a local pool. A failed peer is skipped, a dead worker process is respawned, a failed job is rescheduled with back-off, and a training ring continues on the ranks that are left. Two machines remain single points of failure: the submitter of a parallel script, and the machine of a training ring's rank 0.

A run behaves like a local one. An interrupt sends one cancel frame, which stops the job's process group and its parts on every peer. A dropped connection never kills the job: the daemon logs every output frame, and the submitter can reattach and replay it. Each run ends with a line splitting its time between the script and PipedPeer, and with where its items ran.

4.9. Phones

An Android phone joins from Termux [61] without root. Setup installs a Debian root through `proot-distro`, which uses `proot` [62], with single-user Nix inside it, and falls back to Termux's own Python when that fails. The phone advertises a *chunks-only* role: it takes parallel-map and torch chunks but not whole jobs or training ranks, because the job sandbox would need a second `proot` nested in the first, which the Termux build we tested cannot start. Since closures go only to peers of the same architecture, a phone receives the job's package list, builds an aarch64 environment (mostly fetched from the public binary cache), and accepts the job's chunks once the build is done. Android offers apps neither user namespaces nor cgroups, so the phone's per-job isolation is a `proot` view of the job directory and the store with an address-space limit and a lower CPU priority, which is weaker than the `crun` sandbox used elsewhere.

4.10. Security model

The trust boundary is the cluster: anyone holding the shared secret is a member. Links are authenticated by node keys and the secret, closures are signed by the sending node, and on Linux each job runs in an unprivileged `crun` [60] container with a memory ceiling, without network isolation.

Three gaps remain: TLS on the daemon API is trust on first use; the keys trusted to sign closures are not yet bound to an authenticated identity; and authorisation is one shared secret, so any member can run code on any other. PipedPeer is therefore suited to machines whose owners trust one another, not to open volunteer computing.

5. Implementation

PipedPeer is one Go 1.25 binary, client and daemon, of 46 Go packages, with the interception package embedded as Python sources. It links peers with quic-go, builds environments with Nix flakes, and runs each job in a `crun` sandbox. The Go test suite, run with NumPy, pandas, scikit-learn, joblib and torch installed, passed 1,012 of its 1,017 top-level tests at commit 6bc0ef0. Four were skipped (two need a container runtime, one a Termux environment, one a system-wide Nix), and one failed: a test requiring a cancelled job to stop within one second took 1.56 s; it also failed one of five isolated reruns, so it is timing-sensitive. A separate check fails the build if any of the tests that exercise distribution was skipped, so a missing library cannot turn them into silent passes. A mutation audit disables each of 95 features in turn — among them the sample-weighted gradient average, the refusal to publish a router mapping behind a second NAT, and the check that a signing key belongs to the node offering it — and requires a named test to fail; all 95 were caught [63]. The source code and every measurement record are public [64].

6. Evaluation

We ask five questions. **Q1:** What does interception cost, including when it declines? **Q2:** How much do the environment cache and the closure diff save? **Q3:** Does work cross machines with correct results, how does the division behave on unequal machines, and how does cost grow? **Q4:** Does split training keep accuracy, and when is it faster? **Q5:** What happens between physical machines on different networks, including a phone?

6.1. Setup and method

We use four settings and label every result with its setting.

Plain lab. Three worker daemons in containers on one host, sharing its processor, memory and loopback interface. It measures correctness, overhead

and caching; it cannot measure speedup, because every container competes for the same processor.

Quota lab (emulated). Each container gets a fixed CPU and memory quota and links shaped with `tc netem` to a LAN (0.25 ms one way, 1000 Mbit/s), Wi-Fi (4±2 ms, 80 Mbit/s) or 5G (15±5 ms, 100 Mbit/s). Speedup is one container against several of the same size. Results from this setting are *emulated*: they show how cost grows with one variable at a time, not what physical machines achieve. Some sweeps ran while other work loaded the host (one-minute load 0.4–4 on 15 cores); their growth fits use CPU time, which load barely moves, and their wall times are marked.

Two-site testbed. Two containers on separate private networks, both numbered 192.168.0.0/24, each behind a router container that translates addresses and drops unsolicited inbound packets, with the introducer on a third network standing for the internet. A script runs every command of the project's manual test plan across it and checks each exit code and, where work should cross, that both sites took a share.

Physical machines. Two sets. *Set A*: two laptops in different homes — an AMD Ryzen 7 5800H with an RTX 3060 and about 14 GiB, and an Intel Core i5-13420H with an RTX 3050 and 15.3 GiB — joined over a direct Tailscale path with a 6–8 ms round trip, plus a Samsung Galaxy S20 FE phone (Android 13) under Termux. Neither home offered a reachable introducer (one router refuses port mapping, the other laptop sits behind two NATs), so Set A measures distribution, not NAT traversal. *Set B*: a 20-core 13th-generation Intel Core i9 machine and the Ryzen laptop over Tailscale, training over a home link of about 54 Mbit/s, and a public cloud host serving as introducer.

The labs ran on the two laptops of Set A (Fedora 43 and 44). Lab wall times are medians of three to five runs; training rings are one seed unless stated; interpreter startup is the best of five; sandbox start is the mean of 20 runs after five warm-up runs. Physical-machine figures are single runs or ranges over two or three runs, as stated. Accuracy is the mean over seeds against one device trained on the same data and seeds. MiB is binary, MB decimal.

Test models. Three models (Table 2) keep their batch, learning rate and epochs in every arm.

Table 2: Test models for training accuracy

Model	**Network and optimiser**	**Parameters**	**Samples**	**Batch**	**Epochs (steps)**
MLP-G	MLP 64–512–512–512–1; SGD 0.05, momentum 0.9	559,105	200,000	4096	3 (147)
MLP-C	Same MLP; SGD 0.1, momentum 0.9	559,105	60,000	2048	6 (180)
Transformer	2 layers, width 64, 4 heads, 32 tokens; AdamW	70,211	40,000	256	2 (314)

6.2. Q1: Interception cost

On the Core i5 laptop, startup of a job that imports nothing heavy took 0.117 s with interception against 0.094 s without, a ratio of 1.24× (best of five; Fig. 6). An earlier design that patched NumPy, pandas, torch and joblib at startup measured 20.0×, on a different machine. A continuous-integration check fails any change that raises the ratio above 3×.

A 16-item map whose tail is 0.01 s of work, below the 0.5 s dispatch floor, is declined by the cost model. Its median over five runs was 0.796 s with interception against 0.678 s without, every run's receipt showing no item dispatched: declining costs about 0.12 s per job. A forced run of the same workload did dispatch, so the decline is the model's choice and not an unavailable cluster.

The `crun` sandbox costs 15–21 ms more per job than Bubblewrap across three tasks (means of 20 runs), small beside a warm job of under a second; its device model is what enables GPU passthrough.

6.3. Q2: Environment cache and closure diff

The same trivial job was submitted five times to the plain lab (Fig. 7). The first submission, which also built the environment, took 23.4 s. The second landed on a worker that did not yet hold the closure and took 8.1 s. The next three found it in place, sent only the workspace, and took a median of 0.83 s.

Against a container peer on the same host, adding one package to an environment the peer held sent 7 of 43 store paths, 28.6 MB instead of 206.0 MB

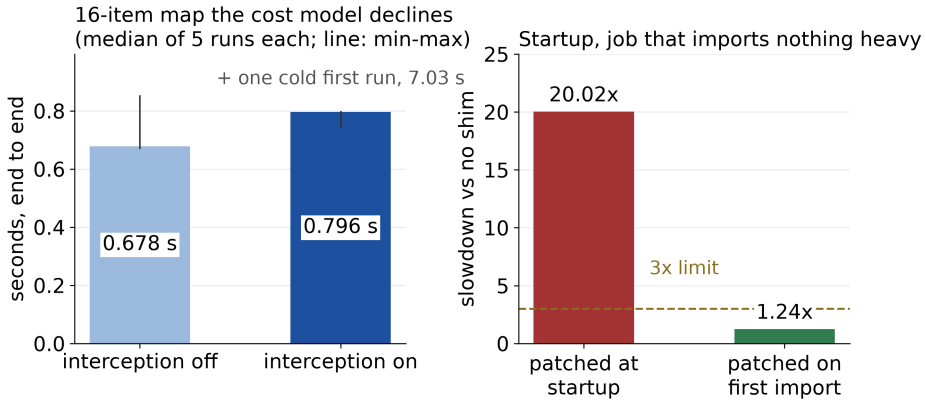


Figure 6: Cost of interception. Left: a declined 16-item map, median and range of five runs (the enabled side's cold first run is excluded from the range). Right: interpreter startup with libraries patched at startup and on first import, best of five.

(86% fewer bytes); a peer fetched 0.45 MB of a 466 MB closure that differed from one it held by one small pure-Python package. In the LAN lab, a dry-run build of an unseen script's environment took 4 s on a container that had built the common packages ahead of time, against 54 s on one that had not.

A defect found by measurement. In an emulated three-node lab, the first version of closure broadcast blocked a cold run for 603 s and a warm run that followed it for 579 s: the upload's reply waited on a synchronous push to every peer, and a peer still importing the last closure was asked again until the 10-minute client timeout. After the push was moved to the background (Section 4.2), the same lab took 137.5 s cold and 62.4 s warm. With one of two nodes held to 800 kbit/s, the first run waited the upload's 60 s limit for the slow node and then started; once the background send finished, a run took 2.1 s with the slow node taking a share.

6.4. Q3: Distribution, division and growth

Correctness. With distribution forced in the plain lab, the node running a 48-item parallel map sent 18 items to three other daemons in each of five runs, with results identical to a local run. A NumPy workload moved its 34 MB singular value decomposition to one worker. Unforced, a 48-item pool benchmark took 0.17 s in plain Python, 0.17 s with a hand-tuned pool and 0.24 s under PipedPeer with 12 items on the cluster: on one shared host, distribution can only add overhead, and here it added 0.07 s.

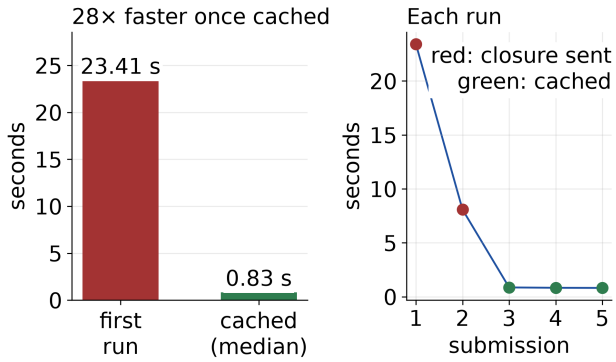


Figure 7: Five submissions of the same environment in the plain lab.

Division on unequal machines. On a quota lab of 8, 2 and 1 CPUs, dividing a map by headcount took 3.6 s, 1.6× slower than not distributing (which puts the local run near 2.3 s). Dividing by measured throughput declined the 1-CPU worker and took 2.9 s: better than the headcount split, but still about 1.3× slower than local. The model predicted a gain that the measurement did not show; at this size the map is better kept on one machine. With one peer held to a tenth of a CPU in a cluster of otherwise equal peers, the slowdown against an equal cluster fell from 1.6× on the first run to 1.08× by the fourth, as the rate estimates converged.

How cost grows (emulated). A sweep varied one quantity at a time around 64 items of about 75 ms and 256 KB each on nodes of 2 CPUs and 768 MiB: payload 1 KB–64 MB, 8–2048 items, per-item cost 10^3–3×10^6 loop iterations, 2–4 nodes, and a speed spread of up to 10× (Table 3). No step's CPU time grew faster than its expected order (log–log slope above the expected exponent by more than 0.15). Below about 10^5 iterations per item the map stayed local, as designed. One peer at a tenth of the speed raised wall time 1.9× where lost capacity alone predicts about 1.4×, a straggler tail. Five percent packet loss moved work back to the submitter (peers 3.0 to 1.7 CPU-s) and raised wall time from 1.6 to 2.9 s; 1–2% cost little. A 256 MB map failed on a 768 MiB submitter for lack of memory.

The step from one node to two adds about 1 CPU-s (2.5 to 3.5), the fixed cost of distributing; from two to four nodes total CPU stays flat.

End to end. Table 4 gives whole-system runs in a LAN lab of four containers on the Ryzen laptop (and a two-container variant sharing its GPU). The

Table 3: Emulated node-count sweep: a 2.5 CPU-second map on equal 2-CPU nodes, median of three runs on a host also carrying other work (one-minute load in the last column), so wall times are inflated unevenly. Not a physical-machine speedup.

Nodes	**Wall (s)**	**vs. one node**	**Total CPU (s)**	**Host load**
1	2.37	1.0×	2.54	2.53
2	1.15	2.1×	3.50	0.96
3	0.95	2.5×	3.55	0.44
4	0.81	2.9×	3.34	0.66

Table 4: End-to-end steps in the LAN lab, first run and cached (one host)

Step	**First run**	**Cached**
Parallel map	43.5 s	17.0 s
Map over a lambda	20.5 s	4.8 s
GPU job submitted from a node without a GPU	—[a]	7.6 s
Two-rank ring on one shared GPU	630.3 s	9.5 s
Script combining every intercepted kind of work	664.6 s	26.7 s
Five idiomatic user scripts[b]	35–126 s each	20–70 s each
Eight jobs at once (6.0 GB of memory in total)	412.0 s, all eight completed	

[a]The recorded first run measured a closure broadcast to nodes without a GPU, which a later change removed, so it is not reported. [b]First run after the common packages were built ahead of time.

containers share one machine, so these are costs, not speedups. Cold times are dominated by building and fetching environments, the GPU steps by downloading CUDA PyTorch.

Failure drills. Table 5 lists drills that remove machines, links and terminals during a run. They exercise rescheduling, re-sharding at an epoch boundary, the cancel frame and the logged output stream. One earlier fault-injection run in the plain lab tested nothing, because no worker held the closure and the map stayed local; we report it as a failed experiment, superseded by the two-site drill in the table's first row. The test suite adds that a parallel map whose every remote part fails still returns the correct result.

Table 5: Failure drills (LAN lab unless stated)

Drill	**Result**
A node's link cut during a 240-item map, two-site testbed	its 53 items re-run on the submitting node once the part overran its predicted 18 s (detected at 81 s); checksum unchanged; map took 83.1 s
A worker killed during a remote job	node reported gone 16 s after the kill
A training rank killed during a run	remaining rank finished all 80 epochs, 90.5 s
A training rank's machine back during a run	rejoined at the next epoch, 4 s after its daemon started
Interrupt during a parallel map	peers stopped within 2 s
Terminal closed during a run	stopped everywhere, as an interrupt
Detach, then attach during and after the run	followed live; replayed in 1.0 s once finished
Submitter's network removed for 200 s during a remote job	job ran once; result back about 15 s after reconnecting
Introducer's address changed, no command run[c]	links held or re-formed with no command; reported 12.1–12.6 s

[c]The harness counts a recovery no earlier than 12 s after the move when the path survives outright, so these readings sit at its floor and do not measure a recovery time.

6.5. Q4: Training

When splitting pays. In the quota lab, a small model with heavy steps on a LAN trained in 9.1 s split across two 2-CPU containers against 11.8 s on one, while a 9 MB model with light steps on Wi-Fi took 80.7 s split against 8.0 s on one container. A GPU widens the gap: on the Ryzen laptop one training job took 1.5 s on its RTX 3060 against 55.0 s on its CPU (one run).

Fast synchronisation. At the default two-rank interval of 32 steps, in-process runs kept validation loss within 1% of one device on every model: MLP-G +0.06%, MLP-C +0.64% and the transformer +0.73% (seeds 0–2), and MLP-C +0.35% and the transformer −1.02% over seeds 0–5 (seeds 3–5 were run for those two only). Single seeds spread from −3.9% to +2.2% across the runs reported here, so the bar holds for the mean, not for every seed. Table 6 gives one lab run (one seed) at this interval: each rank sent 7.6–9.2× fewer bytes than under exact synchronisation, and the ring finished 1.2–1.4× faster than exact on the LAN profile and 1.6–3.1× on Wi-Fi (Fig. 8).

Table 6: Two-rank ring in the quota lab at $H = 32$ (one seed): bytes sent per rank and wall time, against one container

Model	One device (s)	MiB per rank, exact / fast	Exact (s), LAN / Wi-Fi	Fast (s), LAN / Wi-Fi
MLP-G	27.9	81.1 / 10.7	20.0 / 41.8	15.2 / 18.9
MLP-C	16.9	98.9 / 10.7	14.6 / 43.7	10.4 / 14.1
Transformer	26.7	21.7 / 2.4	22.7 / 35.9	19.1 / 22.9

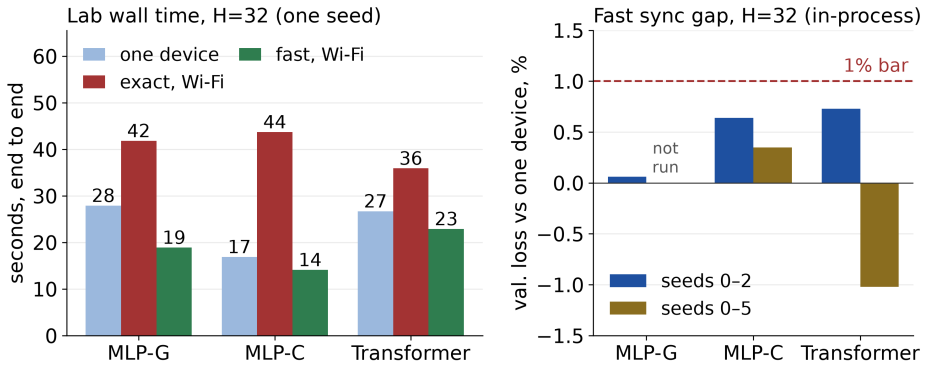


Figure 8: Fast synchronisation at $H = 32$. Left: quota-lab wall time of one device and a two-rank ring under exact and fast synchronisation on Wi-Fi, one seed. Right: in-process validation-loss gap to one device, mean of seeds 0–2 and of seeds 0–5 where run; the dashed line is the 1% bar.

Against one container, the fast ring was faster on every model and profile here (for example MLP-G 15.2 s on LAN against 27.9 s), because each 2-CPU container is CPU-bound; these are equal-quota containers on one host, not physical machines.

At 64 steps, a separate lab run with three seeds per cell also stayed within the bar on both profiles (worst +0.64%), but in-process the transformer failed on seeds 0–2 (+2.13%), so the two-rank default is 32. DiLoCo's outer step left the transformer 21.7% above one device even at one synchronisation per epoch (156 local steps), the longest interval these runs allow.

Many ranks. In-process runs set $H(n)$ as the largest interval that passed on every model: 32 at two ranks, 16 at three, and 4 at four; at eight ranks the transformer failed at every interval down to 2, so no fast interval is used

Table 7: Set A: two laptops in different homes over Tailscale. Script time unless marked "from submission". Sessions 1 and 2 ran different builds.

Run	Result
Mixed CPU script (pool, joblib, NumPy, pandas), one laptop, plain Python, 3 runs	17.4–18.1 s
Same, two laptops, 3 runs	13.3–14.6 s; about half the items on each laptop
Whole job sent to the other laptop	9.1 s; 11.7 s from submission
Small training model, one GPU	4.2 s; validation loss 0.0794
Same, `--ddp 2`, exact (session 1)	37.1 s; loss 0.0797
Same, `--ddp 2`, fast (session 1)	7.4 s; loss 0.0796
Same, `--ddp 2`, automatic (session 1)	32.9 s
Same, `--ddp 2`, fast (session 2)	8.5 s; loss 0.0797
Same, `--ddp 2`, automatic (session 2)	14.6 s; loss 0.0796
Combined CPU+GPU script, from submission: first run / repeat	506.0 s (487.4 s building the environment, mostly CUDA PyTorch) / 17.7 s

for five or more ranks, and fast synchronisation was never measured at five to seven ranks. Of ten methods tested at 4, 8 and 11 ranks, only exact synchronisation with rank-16 PowerSGD [45] and error feedback kept every model within 1% (worst mean +0.18%), with 3.2× fewer bytes than 8-bit values on the perceptrons but 1.7× more on the transformer. This method is measured but not the default.

Adapting the ring. Offered ranks of 16, 8 and 2 CPUs (containers with those quotas on one host), admission took the 16 and the 8, shared the batch 66:34, and ran in 23.6 s against 25.4 s with equal batches. This compares two rings, not a ring with one machine: in the same recorded session, a three-rank ring of those containers took 36.4 s against 6.9 s for the single machine that the admission rule picks, and in other runs on that host two CPU ranks were slower than one (18.9 against 2.8 s; 191.1 against 92.8 s). Throttling one rank's container from 8 CPUs to 1 and back moved the shares from 64:36 to 84:16 and then to 70:30 after it recovered.

6.6. Q5: Physical machines and a phone

Set A. Table 7 lists the two-laptop runs. The mixed CPU script ran faster on two laptops than on one (13.3–14.6 s against 17.4–18.1 s), with about half its items on the second laptop. The small training model did not: one GPU finished first (4.2 s) against every two-laptop configuration. Fast synchronisation was 5.0× faster than exact in session 1 (7.4 against 37.1 s) at about the same loss. Automatic synchronisation was slow in session 1 (32.9 s): that build could not switch a ring fixed by `--ddp 2` to fast mode. The build of session 2 can; it synchronised exactly for 19 steps until its first check at the twentieth synchronisation, then switched (25 synchronisations in all) and took 14.6 s, against 8.5 s for fast synchronisation, whose exact warm-up is 15 of the 147 steps. Exact synchronisation was not repeated in session 2.

Set B. A parallel map took 4.9–5.4 s on the Core i9 machine with the Ryzen laptop as a peer against 5.8–6.4 s alone, with 116 of 256 items on the peer; a job of about 5 s is dominated by setup and serialisation, so the gain is short of what the core counts allow. Training over the 54 Mbit/s home link took 54.9 s on one machine, 69.4 s on two ranks with 16-bit gradients and 56.1 s with 8-bit gradients (final loss 0.0875, 0.0875, 0.0877): exact training over that link was not faster than one machine. With a public cloud host as introducer, the introducer carried 0.2 MB against about 30 MB between the peers, and with the introducer process suspended a known peer was still reached in 0.03 s. These figures were each taken a few times by hand and recorded in the commits that they justified; they were not repeated for this paper.

A phone. With the S20 FE added to Set A, a laptop's CPU script spread over both laptops and the phone gave the phone 8 of 192 items, with an unchanged checksum, after the phone had built the job's aarch64 environment from the binary cache in 27 s. A script training 160 small torch models gave the phone 6 of them, all of which converged; with 32 models the laptops finished before the phone's worker had loaded torch, and the phone took none. Run on the phone alone, the CPU script without its pandas step took 62.4, 62.8 and 66.0 s under Nix's Python in `proot`, and 72.8, 73.1 and 80.8 s under Termux's own Python. The phone therefore contributes a small share of the work; we did not measure whether adding it shortened any run.

Across two NATs. On the two-site testbed, every command of the manual test plan — scripts, parallel maps, remote runs and both training synchroni-

sation modes — ran across the sites and passed its checks, and the cut-link drill of Table 5 recovered the result.

7. Discussion

Measure, then decline. The recurring result is that distribution helps only for work that is large relative to its setup and transfer: the mixed CPU script and the CPU-bound parallel maps gained, while short maps, small models on a GPU, and exact training over a home link did not. Several mechanisms exist in their current form because a first version measured slower than doing nothing: patching libraries at startup (20× startup), dividing work by headcount (1.6× slower than local), returning every rank's gradient, and synchronous closure broadcast (a 600 s stall). The system therefore reports, at the end of each run, how its time was split and where items ran, and it declines when its models predict a loss.

Models versus measurements. The makespan model guarantees only that its *prediction* is no worse than local; on the 8:2:1 lab the measured split was still 1.3× slower than local. The cost model's constants are hand-set. A model that learns per-machine overheads, and a sensitivity study of the placement weights of Eq. (1), are open work.

What generalises. Three ideas are not specific to Python: diffing content-addressed environments by store path between peers; embedding the introducer in every member so that a small group needs no coordination server; and choosing a synchronisation interval per ring size from accuracy measurements rather than from bandwidth alone.

8. Threats to validity and limitations

External validity. The physical-machine results cover two or three machines and one phone, mostly single runs or ranges over two or three runs; one single-sample figure in the project's history was found to be wrong by a factor of nine on repetition, which is why the repeatable benchmark scripts are preferred wherever they exist. No wall-time comparison with Ray, Dask, Lithops or `torchrun` was run, so Table 1 is qualitative. The labs share one host: the plain lab cannot show speedup, and the quota lab emulates quotas and links but not per-core speed, wireless behaviour, campus firewalls or two physical GPUs. NAT traversal was not exercised between physical homes

(Set A ran over Tailscale), and the birthday-collision success rate of Eq. (2) is computed, not measured.

Internal validity. Some emulated sweeps ran on a loaded host; their growth fits use CPU time, but their wall times (Table 3) are inflated unevenly. The 1% accuracy bar is a mean over seeds on three small models, not a guarantee for every seed or for larger models.

System limitations. Only Python on Linux is supported, with NVIDIA GPUs in practice. The submitter of a parallel script and the machine of a training ring's rank 0 are single points of failure. Two machines that both lack a reachable introducer cannot meet without an overlay network, because there is no relay by design. Phones run chunks only, under a weaker sandbox. The security gaps of Section 4.10 restrict use to mutually trusting machines.

9. Conclusion and future work

PipedPeer runs unmodified Python on other machines without a configured cluster: it ships environments as closures diffed by store path, substitutes the parallel primitives a script already uses, divides work by measured speed, and joins machines across networks over direct authenticated links with no relay and no privileged node. On the machines we had, it made CPU-bound work faster on two machines than on one and kept training accuracy within 1% at a fraction of the bytes, and it was slower than one machine for short jobs and for small models on a GPU. Future work is to bind signing keys to authenticated identities and require TLS; to let a parallel script and a training ring survive their submitter and rank 0; to run whole jobs on phones; to add locality- and topology-aware placement; and to run a controlled, repeated evaluation on more physical machines against Ray, Dask and Lithops.

CRediT authorship contribution statement

[AUTHORS: state each author's roles using the CRediT taxonomy (Conceptualization, Software, Investigation, Writing – original draft, Supervision, . . .). Only the authors know this.]

Declaration of competing interest

[AUTHORS: confirm: "The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper."]

Funding

[AUTHORS: state funding, or "This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors."]

Declaration of generative AI use

[AUTHORS: Elsevier requires this statement. The manuscript and parts of the software were prepared with the help of an AI assistant (Claude, Anthropic); state how, and confirm that the authors reviewed and edited the content and take full responsibility for it.]

Data availability

The source code, benchmark scripts and every measurement record cited in this paper are available in the project repository [64], under `docs/benchmarks`.

Acknowledgements

[AUTHORS: optional.]

References

[1] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, I. Stoica, Spark: Cluster computing with working sets, in: Proc. 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud), 2010.

[2] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, I. Stoica, Ray: A distributed framework for emerging AI applications, in: Proc. 13th USENIX Symp. on Operating Systems Design and Implementation (OSDI), 2018, pp. 561–577.

[3] M. Rocklin, Dask: Parallel computation with blocked algorithms and task scheduling, in: Proc. 14th Python in Science Conf. (SciPy), 2015, pp. 126–132.

[4] D. Petersohn, S. Macke, D. Xin, W. Ma, D. Lee, X. Mo, J. E. Gonzalez, J. M. Hellerstein, A. D. Joseph, A. Parameswaran, Towards scalable dataframe systems, Proc. VLDB Endow. 13 (12) (2020) 2033–2046.

[5] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, J. Wilkes, Borg, Omega, and Kubernetes, ACM Queue 14 (1) (2016) 70–93.

[6] M. J. Litzkow, M. Livny, M. W. Mutka, Condor – a hunter of idle workstations, in: Proc. 8th Int. Conf. on Distributed Computing Systems (ICDCS), 1988, pp. 104–111.

[7] J. Zhi, R. Wang, J. Clune, K. O. Stanley, Fiber: A platform for efficient development and distributed training for reinforcement learning and population-based methods, arXiv:2003.11164 (2020).

[8] E. Jonas, Q. Pu, S. Venkataraman, I. Stoica, B. Recht, Occupy the cloud: Distributed computing for the 99%, in: Proc. ACM Symp. on Cloud Computing (SoCC), 2017, pp. 445–451.

[9] J. Sampé, P. García-López, M. Sánchez-Artigas, G. París, D. Barcelona-Pons, D. Arroyo-Pinto, Toward multicloud access transparency in serverless computing, IEEE Software 38 (1) (2021) 68–74.

[10] Y. Babuji, A. Woodard, Z. Li, D. S. Katz, B. Clifford, R. Kumar, L. Lacinski, R. Chard, J. M. Wozniak, I. Foster, M. Wilde, K. Chard, Parsl: Pervasive parallel programming in Python, in: Proc. 28th Int. Symp. on High-Performance Parallel and Distributed Computing (HPDC), 2019, pp. 25–36.

[11] R. Chard, Y. Babuji, Z. Li, T. Skluzacek, A. Woodard, B. Blaiszik, I. Foster, K. Chard, funcX: A federated function serving fabric for science, in: Proc. 29th Int. Symp. on High-Performance Parallel and Distributed Computing (HPDC), 2020, pp. 65–76.

[12] E. Dolstra, M. de Jonge, E. Visser, Nix: A safe and policy-free system for software deployment, in: Proc. 18th Large Installation System Administration Conf. (LISA), 2004, pp. 79–92.

[13] E. Dolstra, The Purely Functional Software Deployment Model, Ph.D. thesis, Utrecht University, 2006.

[14] L. Courtès, Functional package management with Guix, in: Proc. European Lisp Symposium (ELS), 2013.

[15] T. Gamblin, M. LeGendre, M. R. Collette, G. L. Lee, A. Moody, B. R. de Supinski, S. Futral, The Spack package manager: Bringing order to HPC software chaos, in: Proc. Int. Conf. for High Performance Computing, Networking, Storage and Analysis (SC), 2015.

[16] D. Merkel, Docker: Lightweight Linux containers for consistent development and deployment, Linux Journal 2014 (239) (2014).

[17] G. M. Kurtzer, V. Sochat, M. W. Bauer, Singularity: Scientific containers for mobility of compute, PLoS ONE 12 (5) (2017) e0177459.

[18] D. P. Anderson, J. Cobb, E. Korpela, M. Lebofsky, D. Werthimer, SETI@home: An experiment in public-resource computing, Commun. ACM 45 (11) (2002) 56–61.

[19] D. P. Anderson, BOINC: A system for public-resource computing and storage, in: Proc. 5th IEEE/ACM Int. Workshop on Grid Computing, 2004, pp. 4–10.

[20] E. Cuervo, A. Balasubramanian, D. Cho, A. Wolman, S. Saroiu, R. Chandra, P. Bahl, MAUI: Making smartphones last longer with code offload, in: Proc. 8th Int. Conf. on Mobile Systems, Applications, and Services (MobiSys), 2010, pp. 49–62.

[21] B.-G. Chun, S. Ihm, P. Maniatis, M. Naik, A. Patti, CloneCloud: Elastic execution between mobile device and cloud, in: Proc. 6th European Conf. on Computer Systems (EuroSys), 2011, pp. 301–314.

[22] M. Y. Arslan, I. Singh, S. Singh, H. V. Madhyastha, K. Sundaresan, S. V. Krishnamurthy, Computing while charging: Building a distributed computing infrastructure using smartphones, in: Proc. 8th Int. Conf. on Emerging Networking Experiments and Technologies (CoNEXT), 2012, pp. 193–204.

[23] N. Fernando, S. W. Loke, W. Rahayu, Honeybee: A programming framework for mobile crowd computing, in: Proc. 9th Int. Conf. on Mobile and Ubiquitous Systems (MobiQuitous), 2012, pp. 224–236.

[24] M. Petit-Huguenin, G. Salgueiro, J. Rosenberg, D. Wing, R. Mahy, P. Matthews, Session Traversal Utilities for NAT (STUN), RFC 8489, IETF, 2020.

[25] D. Wing, S. Cheshire, M. Boucadair, R. Penno, P. Selkirk, Port Control Protocol (PCP), RFC 6887, IETF, 2013.

[26] S. Cheshire, M. Krochmal, NAT Port Mapping Protocol (NAT-PMP), RFC 6886, IETF, 2013.

[27] UPnP Forum, Internet Gateway Device (IGD) V2.0, Standardized DCP, 2010.

[28] A. Keranen, C. Holmberg, J. Rosenberg, Interactive Connectivity Establishment (ICE): A protocol for network address translator (NAT) traversal, RFC 8445, IETF, 2018.

[29] B. Ford, P. Srisuresh, D. Kegel, Peer-to-peer communication across network address translators, in: Proc. USENIX Annual Technical Conf., 2005, pp. 179–192.

[30] D. Anderson, How NAT traversal works, Tailscale blog, 2020. URL `https://tailscale.com/blog/how-nat-traversal-works` (accessed 16 October 2026).

[31] J. Iyengar, M. Thomson, QUIC: A UDP-based multiplexed and secure transport, RFC 9000, IETF, 2021.

[32] J. A. Donenfeld, WireGuard: Next generation kernel network tunnel, in: Proc. Network and Distributed System Security Symp. (NDSS), 2017.

[33] R. L. Graham, Bounds on multiprocessing timing anomalies, SIAM J. Appl. Math. 17 (2) (1969) 416–429.

[34] D. S. Hochbaum, D. B. Shmoys, A polynomial approximation scheme for scheduling on uniform processors: Using the dual approximation approach, SIAM J. Comput. 17 (3) (1988) 539–551.

[35] J. Dean, S. Ghemawat, MapReduce: Simplified data processing on large clusters, in: Proc. 6th USENIX Symp. on Operating Systems Design and Implementation (OSDI), 2004, pp. 137–150.

[36] M. Zaharia, A. Konwinski, A. D. Joseph, R. Katz, I. Stoica, Improving MapReduce performance in heterogeneous environments, in: Proc. 8th USENIX Symp. on Operating Systems Design and Implementation (OSDI), 2008, pp. 29–42.

[37] Y. Deng, Z. Chen, X. Chen, Y. Fang, Task offloading in multi-hop relay-aided multi-access edge computing, IEEE Trans. Veh. Technol. 72 (1) (2023) 1372–1376.

[38] D. Yong, R. Liu, X. Jia, Y. Gu, Joint optimization of multi-user partial offloading strategy and resource allocation strategy in D2D-enabled MEC, Sensors 23 (5) (2023) 2565.

[39] H. He, X. Yang, X. Mi, H. Shen, X. Liao, Multi-agent deep reinforcement learning based dynamic task offloading in a device-to-device mobile-edge computing network to minimize average task delay with deadline constraints, Sensors 24 (16) (2024) 5141.

[40] Z. Zhao, J. Perazzone, G. Verma, S. Segarra, Congestion-aware distributed task offloading in wireless multi-hop networks using graph neural networks, in: Proc. IEEE Int. Conf. on Acoustics, Speech and Signal Processing (ICASSP), 2024, pp. 8951–8955.

[41] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, S. Chintala, PyTorch distributed: Experiences on accelerating data parallel training, Proc. VLDB Endow. 13 (12) (2020) 3005–3018.

[42] A. Sergeev, M. Del Balso, Horovod: Fast and easy distributed deep learning in TensorFlow, arXiv:1802.05799 (2018).

[43] T. Dettmers, 8-bit approximations for parallelism in deep learning, in: Proc. Int. Conf. on Learning Representations (ICLR), 2016.

[44] F. Seide, H. Fu, J. Droppo, G. Li, D. Yu, 1-bit stochastic gradient descent and its application to data-parallel distributed training of speech DNNs, in: Proc. Interspeech, 2014, pp. 1058–1062.

[45] T. Vogels, S. P. Karimireddy, M. Jaggi, PowerSGD: Practical low-rank gradient compression for distributed optimization, in: Advances in Neural Information Processing Systems (NeurIPS), 2019.

[46] S. U. Stich, Local SGD converges fast and communicates little, in: Proc. Int. Conf. on Learning Representations (ICLR), 2019.

[47] T. Lin, S. U. Stich, K. K. Patel, M. Jaggi, Don't use large mini-batches, use local SGD, in: Proc. Int. Conf. on Learning Representations (ICLR), 2020.

[48] B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proc. 20th Int. Conf. on Artificial Intelligence and Statistics (AISTATS), 2017, pp. 1273–1282.

[49] A. Douillard, Q. Feng, A. A. Rusu, R. Chhaparia, Y. Donchev, A. Kuncoro, M. Ranzato, A. Szlam, J. Shen, DiLoCo: Distributed low-communication training of language models, arXiv:2311.08105 (2023).

[50] M. Ryabinin, A. Gusev, Towards crowdsourced training of large neural networks using decentralized mixture-of-experts, in: Advances in Neural Information Processing Systems (NeurIPS), 2020.

[51] M. Diskin, A. Bukhtiyarov, M. Ryabinin, L. Saulnier, Q. Lhoest, A. Sinitsin, D. Popov, D. Pyrkin, M. Kashirin, A. Borzunov, A. Villanova del Moral, D. Mazur, I. Kobelev, Y. Jernite, T. Wolf, G. Pekhimenko, Distributed deep learning in open collaborations, in: Advances in Neural Information Processing Systems (NeurIPS), 2021.

[52] A. Borzunov, D. Baranchuk, T. Dettmers, M. Ryabinin, Y. Belkada, A. Chumachenko, P. Samygin, C. Raffel, Petals: Collaborative inference and fine-tuning of large models, in: Proc. 61st Annual Meeting of the Association for Computational Linguistics: System Demonstrations (ACL), 2023, pp. 558–568.

[53] J. Spenger, Programming Models for Failure-Transparent Distributed Systems, Ph.D. thesis, KTH Royal Institute of Technology, Stockholm, 2025.

[54] D. G. Thaler, C. V. Ravishankar, Using name-based mappings to increase hit rates, IEEE/ACM Trans. Netw. 6 (1) (1998) 1–14.

[55] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, B.-Y. Yang, High-speed high-security signatures, J. Cryptogr. Eng. 2 (2) (2012) 77–89.

[56] E. Rescorla, Keying material exporters for Transport Layer Security (TLS), RFC 5705, IETF, 2010.

[57] H. Krawczyk, M. Bellare, R. Canetti, HMAC: Keyed-hashing for message authentication, RFC 2104, IETF, 1997.

[58] E. Rescorla, The Transport Layer Security (TLS) protocol version 1.3, RFC 8446, IETF, 2018.

[59] cloudpickle: Extended pickling support for Python objects, 2015. URL `https://github.com/cloudpipe/cloudpickle` (accessed 16 October 2026).

[60] crun: A fast and lightweight fully featured OCI runtime and C library for running containers, 2017. URL `https://github.com/containers/crun` (accessed 16 October 2026).

[61] Termux: Android terminal emulator and Linux environment, 2015. URL `https://termux.dev` (accessed 16 October 2026).

[62] PRoot: chroot, mount –bind, and binfmt_misc without privilege/setup for Linux, 2010. URL `https://proot-me.github.io` (accessed 16 October 2026).

[63] Y. Jia, M. Harman, An analysis and survey of the development of mutation testing, IEEE Trans. Softw. Eng. 37 (5) (2011) 649–678.

[64] Y. Vaviya, V. Yadav, S. Gupta, PipedPeer: source code and measurement records. URL `https://github.com/vinayakyadav2709/Pipedpeer` (accessed 16 October 2026).